#17 - Retrieve rồi Rerank: Nâng độ chính xác tìm kiếm ngữ nghĩa với Bi-Encoder và Cross-Encoder

Trong series này chúng ta đã đi qua từng bước để tạo ra một chatbot RAG, từ chunking, embedding, biết cách triển khai vector database Qdrant, tạo collection và tìm kiếm ngữ nghĩa trên collection đó. Trong bài viết này tôi sẽ mang đến cho các bạn một khái niệm nữa trong phạm trù RAG là Rerank, đồng thời demo cho các bạn xem Rerank ảnh hưởng tới chất lượng retrieve như thế nào.

Bài viết này dành cho ai?

Các lập trình viên/kỹ sư AI đã biết khái niệm embedding cơ bản, muốn tự tay dựng pipeline retrieve rồi rerank để nâng chất lượng tìm kiếm ngữ nghĩa (semantic search, RAG); không yêu cầu kinh nghiệm huấn luyện mô hình chuyên sâu.

Mục tiêu:

- Hiểu bản chất kiến trúc Bi-Encoder và Cross-Encoder, vì sao kết hợp hai giai đoạn retrieve rồi rerank lại chính xác hơn dùng một mình bi-encoder
- Tự cài đặt `sentence-transformers` với model bge-m3 và bge-reranker-v2-m3 của BAAI để dựng pipeline retrieve top-20 rồi rerank top-5 kết nối Qdrant
- Đo lường chất lượng pipeline bằng bảng so sánh thứ hạng trước/sau và các chỉ số Hit@k, MRR trên 10 truy vấn với ground-truth thật
- Biết cách batch inference, debug OOM/latency, và nhận diện khi nào chưa cần thêm bước rerank

Tác giả: Kỹ sư AI Đinh Văn Vinh
AI Guru x TiniX

MỤC LỤC

BẢNG THUẬT NGỮ 4

1 CƠ SỞ LÝ THUYẾT 5
1.1 Kiến trúc Bi-Encoder và Cross-Encoder 5
1.2 Vì sao Cross-Encoder chính xác hơn nhưng chậm hơn 6
1.2.1 *Độ phức tạp tính toán* 6
1.2.2 *Khác biệt hàm mất mát khi huấn luyện* 7
1.2.3 *Tóm gọn trục đánh đổi* 8
1.3 Two-stage Retrieval Pipeline 8
1.4 Metric đánh giá 9

2 CHUẨN BỊ MÔI TRƯỜNG 10
2.1 Cài đặt Python hoặc uv 10
2.2 Driver và CUDA cho GPU NVIDIA 10
2.3 Cài đặt thư viện Python 11

3 XÂY DỰNG MÃ NGUỒN PIPELINE 12
3.1 Data Class 13
3.2 Retriever và Reranker 13
3.3 Main function 15

4 ĐÁNH GIÁ KẾT QUẢ 17
4.1 Môi trường kiểm thử 17
4.2 Kết quả kiểm thử 18

5 VẤN ĐỀ BỘ NHỚ VÀ ĐỘ TRỄ CỦA PIPELINE 22
5.1 Vì sao rerank dễ gặp OOM 22
5.2 Ước lượng VRAM và chọn batch_size phù hợp 23

5.3 Tối ưu latency 24

6 KẾT LUẬN 25

NGUỒN THAM KHẢO 27

AI Guru

AI Guru

AI Guru

Bảng thuật ngữ

Thuật ngữ	Nghĩa tiếng Việt	Giải thích ngắn
Embedding	Biểu diễn nhúng	Kỹ thuật chuyển đổi văn bản, hình ảnh hoặc dữ liệu thành chuỗi số (vector) trong không gian nhiều chiều để máy tính so sánh độ tương đồng ngữ nghĩa.
Retrieval	Truy xuất	Bước đầu trong pipeline tìm kiếm ngữ nghĩa, dùng vector embedding để lấy nhanh một tập ứng viên có khả năng liên quan cao nhất từ toàn bộ collection.
Rerank	Xếp hạng lại	Bước chấm lại điểm liên quan cho tập ứng viên đã truy xuất bằng một mô hình chính xác hơn nhưng chậm hơn, nhằm sắp xếp lại thứ tự cho đúng hơn trước khi trả về người dùng.

1. Cơ sở lý thuyết

1.1 Kiến trúc Bi-Encoder và Cross-Encoder

Cả Bi-Encoder và Cross-Encoder đều dựa trên Embedding model nền Transformer, nhưng cách chúng xử lý cặp document lại khác nhau hoàn toàn.

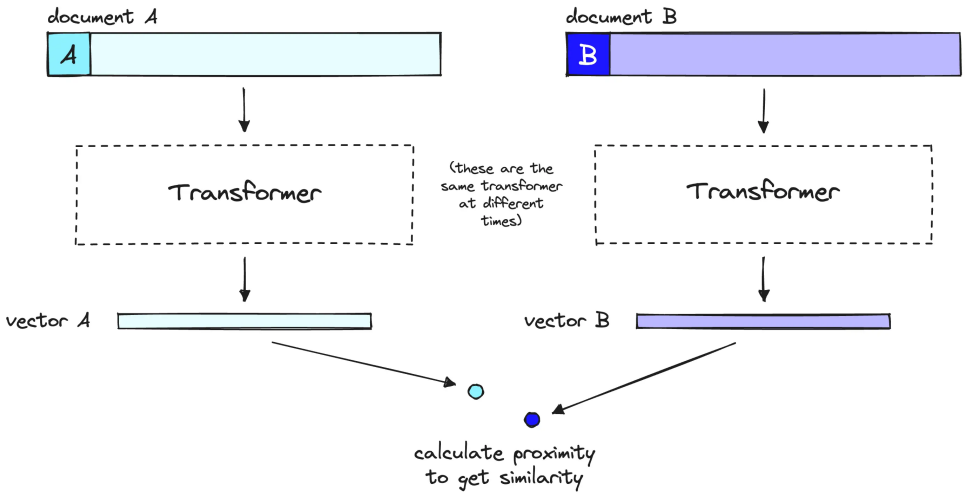


Figure 1: Kiến trúc Bi-Encoder: mỗi document đi qua cùng một Transformer ở hai lượt tách biệt, cho ra hai vector rồi mới so khớp

Trong Hình 1, document A và document B lần lượt đi qua cùng một Transformer nhưng ở hai lượt tách biệt, mỗi lượt cho ra một vector riêng. Việc so khớp chỉ diễn ra sau cùng, ở bước tính độ tương đồng giữa hai vector đó.

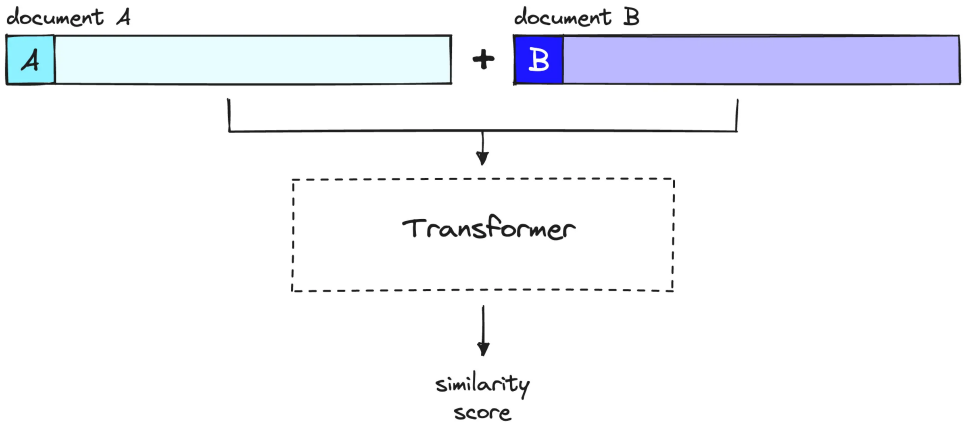


Figure 2: Kiến trúc Cross-Encoder: hai document được nối thành một input duy nhất trước khi đưa qua Transformer

Ngược lại, trong Hình 2, document A và document B được nối thành một chuỗi duy nhất ngay từ đầu vào, đi qua Transformer chỉ một lượt và trả thẳng ra một similarity score.

Bi-Encoder Mã hoá query và document thành hai vector Embedding hoàn toàn độc lập

với nhau, rồi so khớp bằng một phép đo tương đồng như cosine similarity. Vì document được encode tách rời khỏi query, bạn có thể tính vector trước cho toàn bộ tập document và lưu lại, sau đó chỉ cần encode query một lần tại thời điểm tìm kiếm.

Cross-Encoder Nhận đồng thời cả query và document làm một input duy nhất, cho đi qua mô hình transformer với self-attention đầy đủ giữa hai chuỗi, rồi trả về một điểm relevance duy nhất cho cặp đó. Vì phải chạy lại toàn bộ mô hình cho từng cặp query-document, Cross-Encoder không thể tính trước hay lưu lại kết quả như Bi-Encoder.

1.2 Vì sao Cross-Encoder chính xác hơn nhưng chậm hơn

Để hiểu được vì sao Cross-Encoder chính xác hơn so với Bi-Encoder nhưng đồng thời phải đánh đổi bằng tốc độ, chúng ta nên hiểu được bản chất toán học đằng sau chúng.

1.2.1 Độ phức tạp tính toán

Điểm tương đồng của Bi-Encoder được tính bằng tích trong giữa hai vector Embedding:

$$s(q, d) = \phi(q)^\top \psi(d), \qquad \phi(q), \psi(d) \in \mathbb{R}^{d_e}$$

Trong đó ϕ và ψ lần lượt là mạng encode query và document, còn d_e là số chiều của không gian Embedding đầu ra.

Mỗi document được encode độc lập và offline, chỉ một lần duy nhất, rồi lưu vào một chỉ mục tìm kiếm gần đúng như HNSW, IVF-PQ hay ScaNN. Tại thời điểm truy vấn, chi phí gồm hai phần: encode query tốn $O(L_q^2 d_{\text{model}} + L_q d_{\text{model}}^2)$ cho một lượt forward pass transformer chuẩn, với L_q là số token của query và d_{model} là số chiều hidden của transformer, còn tra cứu trong chỉ mục chỉ tốn khoảng $O(\log N)$ với HNSW hoặc $O(\sqrt{N})$ với IVF, thay vì phải duyệt tuyến tính qua toàn bộ N document trong tập. Tổng chi phí tại thời điểm truy vấn gần như không đổi theo kích thước tập document, vì phần tính toán nặng nhất đã được dồn hết sang giai đoạn indexing từ trước.

Đánh đổi cho tốc độ này nằm ở chỗ điểm tương đồng chỉ là một phép tích trong trong không gian d_e chiều cố định. Về bản chất toán học, ma trận điểm tương đồng giữa mọi cặp query-document luôn bị giới hạn hạng bởi d_e, nên Bi-Encoder không thể biểu diễn được mọi hàm mức độ liên quan tuỳ ý, dù huấn luyện tốt tới đâu.

Cross-Encoder không tách embedding của query và document mà nối chúng thành một chuỗi input duy nhất rồi đưa qua transformer:

$$s(q, d) = W \cdot \text{CLS}\Big(f_\theta\big([\text{CLS}]; q; [\text{SEP}]; d; [\text{SEP}]\big)\Big)$$

Trong đó f_θ là mô hình transformer với tham số θ, hai token [CLS] và [SEP] đánh dấu điểm bắt đầu chuỗi và ranh giới giữa query với document, còn W là ma trận trọng số ánh xạ vector CLS đầu ra thành một điểm số duy nhất.

Với N ứng viên cần rerank, chi phí tính toán là:

$$O\big(N \cdot L^2 \cdot d_{\text{model}} \cdot n_{\text{layers}}\big), \qquad L = L_q + L_d$$

Trong đó L là tổng độ dài query cộng document, còn n_{layers} là số layer của transformer.

Điểm mấu chốt nằm ở chỗ L^2 không tách được thành $L_q^2 + L_d^2$, vì trong self-attention, ma trận $QK^\top$ kích thước $L \times L$ chứa cả các block chéo giữa query và document. Chính các block chéo này mang thông tin tương tác chi tiết mà Bi-Encoder không có, nhưng cũng là lý do khiến bạn không thể tách q và d ra để tính riêng rồi ghép lại như Bi-Encoder.

Về con số cụ thể, nhiều báo cáo benchmark thực tế cho thấy với BERT-base gồm 12 layer, $d_{\text{model}} = 768$ và $L = 256$ token, rerank $N = 100$ document trên GPU tốn khoảng 30–80 ms tuỳ cách batch. Trong khi đó Bi-Encoder chỉ mất khoảng 5–10 ms để encode một query, cộng thêm dưới 5 ms để tra cứu chỉ mục trên hàng triệu vector. Chênh lệch bậc 5–10 lần là phổ biến, thậm chí lên tới cả trăm lần nếu N lớn hoặc pipeline không tận dụng batch tốt.

1.2.2 Khác biệt hàm mất mát khi huấn luyện

Ngoài kiến trúc, cách hai mô hình được huấn luyện cũng quyết định phần lớn khoảng cách chất lượng giữa chúng. Bi-Encoder thường được huấn luyện bằng contrastive loss, phổ biến nhất là InfoNCE:

$$\mathcal{L} = -\log \frac{\exp\big(\phi(q)^\top \psi(d^+)/\tau\big)}{\displaystyle\sum_{d_i \in \{d^+\} \cup D^-} \exp\big(\phi(q)^\top \psi(d_i)/\tau\big)}$$

Trong đó d^+ là document đúng ứng với query q, D^- là tập document negative dùng để tương phản, còn τ là hệ số điều chỉnh độ sắc nét của phân phối xác suất đầu ra.

Mục tiêu của hàm mất mát này là ép toàn bộ không gian Embedding tổ chức theo một cấu trúc hình học nhất quán, sao cho cặp liên quan luôn gần nhau và cặp không liên quan luôn xa nhau, theo đúng một phép đo cố định như cosine similarity áp dụng đồng loạt cho mọi cặp. Ràng buộc toàn cục này chính là nguồn gốc thêm của giới hạn hạng đã nhắc ở trên.

Cross-Encoder thì huấn luyện như bài toán phân loại nhị phân hoặc pairwise ranking:

$$\mathcal{L} = \text{BCE}\left(f_\theta(q, d), y\right) \quad \text{hoặc} \quad \mathcal{L} = \max\left(0,\ m - s(q, d^+) + s(q, d^-)\right)$$

Trong đó y là nhãn đúng/sai của cặp (q, d), còn m là margin, khoảng cách tối thiểu yêu cầu giữa điểm số của cặp dương (q, d^+) và cặp âm (q, d^-).

Vì mỗi cặp query-document được đánh giá độc lập, Cross-Encoder không bị ràng buộc phải tổ chức bất kỳ không gian vector toàn cục nào, mô hình được tự do học bất kỳ hàm phi tuyến nào miễn khớp label. Đây là lý do trên nhiều benchmark truy hồi thông

tin như MS MARCO hay BEIR, Cross-Encoder thường cho điểm NDCG@10 cao hơn Bi-Encoder cùng kích thước một khoảng đáng kể.

1.2.3 Tóm gọn trục đánh đổi

Tiêu chí	Bi-Encoder	Cross-Encoder
Khi nào q và d tương tác	Chỉ ở bước tích trong cuối cùng	Ngay từ layer đầu tiên
Precompute document	Có, một vector mỗi document	Không
Độ phức tạp truy vấn	$O(\log N)$ hoặc $O(\sqrt{N})$	$O(N \cdot L^2)$
Độ chính xác	Thấp hơn	Cao hơn

Trên thực tế, không ai chạy Cross-Encoder trên hàng triệu document cả, nó quá chậm. Thay vào đó, quy trình 2 bước phổ biến là lọc thô bằng Bi-Encoder sau đó xếp hạng lại bằng Cross-Encoder, thực chất là cách khai thác trục đánh đổi này.

Chúng ta dùng tốc độ của Bi-Encoder để cắt giảm số lượng ứng viên từ N xuống còn vài chục, rồi dùng Cross-Encoder, vốn chỉ khả thi khi số lượng ứng viên đã nhỏ, để lấy lại phần độ chính xác còn thiếu.

1.3 Two-stage Retrieval Pipeline

Pipeline nói trên có thể vẽ thành hai giai đoạn cụ thể, thể hiện trong Hình 3.

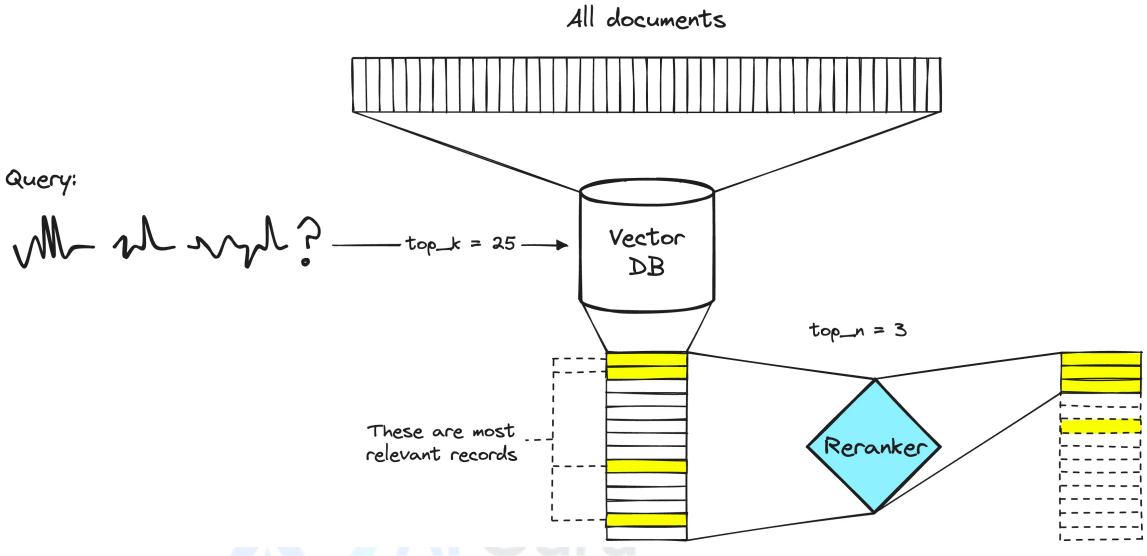


Figure 3: Pipeline hai giai đoạn: retrieve bằng chỉ mục vector rồi rerank bằng Cross-Encoder

Giai đoạn 1: retrieve Bi-Encoder encode toàn bộ document trong tập dữ liệu thành vector từ trước, lưu vào một chỉ mục vector. Khi có query mới, hệ thống chỉ cần encode

query một lần, so khớp với toàn bộ vector đã lưu, rồi lấy ra $k_{retrieval}$ document có điểm tương đồng cao nhất.

Giai đoạn 2: rerank Cross-Encoder nhận lần lượt từng cặp gồm query và một trong $k_{retrieval}$ document ứng viên, đọc đồng thời cả hai để tính điểm relevance chính xác hơn, rồi sắp xếp lại và chỉ giữ k_{rerank} document xếp hạng cao nhất trả về cho người dùng.

1.4 Metric đánh giá

Để biết pipeline có thực sự cải thiện chất lượng tìm kiếm hay không, tôi cần một con số cụ thể thay vì chỉ nhìn qua vài query rồi đánh giá cảm tính. Hit@k và MRR là hai chỉ số đơn giản, tính tay được và đủ dùng trong phạm vi bài viết này.

Hit@k Tỉ lệ phần trăm số query có document đúng xuất hiện trong top-k kết quả trả về, tính trên toàn bộ tập query đánh giá.

MRR Viết tắt của Mean Reciprocal Rank, là giá trị trung bình của nghịch đảo vị trí xuất hiện đầu tiên của document đúng trong danh sách kết quả, tính trên toàn bộ tập query đánh giá.

Để hình dung rõ hơn, giả sử tôi có 4 query thử nghiệm, mỗi query chỉ có đúng một document liên quan, và pipeline trả về vị trí của document đó trong kết quả như bảng dưới đây.

Query	Rank của document đúng
Q1	1
Q2	3
Q3	Không xuất hiện trong top-5
Q4	2

Với $k = 3$, Hit@3 chỉ tính những query có document đúng nằm ở rank nhỏ hơn hoặc bằng 3, tức Q1, Q2 và Q4 đạt, riêng Q3 không đạt vì nằm ngoài top-5. Hit@3 trong trường hợp này bằng $3/4 = 75\%$.

MRR tính theo nghịch đảo rank của từng query, quy ước những query không tìm thấy document đúng nhận giá trị 0:

$$\text{MRR} = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{\text{rank}_i} = \frac{1}{4}\left(\frac{1}{1} + \frac{1}{3} + 0 + \frac{1}{2}\right) \approx 0.46$$

Cùng một tập kết quả nhưng hai chỉ số phản ánh hai góc nhìn khác nhau. Hit@3 chỉ quan tâm document đúng có lọt top-3 hay không, còn MRR phạt nặng hơn khi document đúng nằm càng xa vị trí đầu, kể cả khi vẫn còn lọt top-k.

2. Chuẩn bị môi trường

2.1 Cài đặt Python hoặc uv

Pipeline trong bài dùng Python, nên máy bạn cần có Python từ bản 3.9 trở lên. Ngoài cách cài Python truyền thống, tôi khuyến khích dùng `uv`, một công cụ quản lý Python và thư viện viết bằng Rust, tự tải đúng bản Python cần thiết mà không cần bạn cài tay trước.

```
# Kiểm tra phiên bản Python và uv đã có trên máy chưa
python3 --version && uv --version
```

Kết quả trên máy tôi:

```
Python 3.9.6
uv 0.10.6 (Homebrew 2026-02-24)
```

Máy tôi có sẵn Python 3.9.6 do hệ điều hành cài kèm, nhưng bản này khá cũ so với yêu cầu của các thư viện AI hiện tại. Vì đã có `uv`, tôi để `uv` tự tải và quản lý một bản Python mới hơn thay vì nâng cấp Python hệ thống, tránh xung đột với các phần mềm khác đang phụ thuộc vào Python 3.9.6.

macOS Cài `uv` bằng Homebrew qua lệnh `brew install uv`, hoặc dùng script cài đặt chính thức tại `https://astral.sh/uv`. Nếu chỉ cần Python thuần, chạy `brew install python@3.12`.

Windows Mở PowerShell và chạy `winget install astral-sh.uv`, hoặc tải trình cài đặt Python bản mới nhất tại `https://python.org`.

Linux Trên Ubuntu hoặc Debian, cài Python qua `sudo apt install -y python3 python3-venv`, hoặc cài `uv` bằng script chính thức tại `https://astral.sh/uv`.

2.2 Driver và CUDA cho GPU NVIDIA

Phần này chỉ áp dụng nếu bạn muốn chạy pipeline trên GPU NVIDIA để tăng tốc. Với các cấu hình còn lại, bạn có thể bỏ qua mục này.

macOS, cả Apple Silicon lẫn Intel Không cần cài thêm gì. Apple Silicon dùng sẵn backend MPS của PyTorch để tăng tốc, còn Intel chạy thẳng trên CPU.

Chạy CPU thuần trên Windows hoặc Linux Cũng không cần cài thêm gì, thư viện sẽ tự dùng CPU nếu không phát hiện GPU tương thích.

Windows hoặc Linux dùng GPU NVIDIA Bắt buộc cài NVIDIA driver đúng dòng GPU, sau đó cài CUDA Toolkit ở bản tương thích với driver đó. Hai điều kiện tương thích cần đảm bảo: driver phải hỗ trợ đúng dòng GPU của bạn, và bản CUDA Toolkit phải nằm trong danh sách driver đó hỗ trợ, tra tại bảng tương thích chính

thức của NVIDIA.

> **LƯU Ý QUAN TRỌNG** Sau khi cài xong cả driver và CUDA Toolkit trên Windows hoặc Linux, bạn bắt buộc phải khởi động lại máy. Hệ điều hành chỉ nạp driver mới vào nhân hệ thống sau khi restart, nên nếu bỏ qua bước này, các lệnh kiểm tra GPU phía sau vẫn báo không tìm thấy thiết bị dù đã cài đúng.

Sau khi restart, bạn kiểm tra driver và CUDA đã được hệ thống nhận đúng chưa bằng hai lệnh sau, chạy trong Command Prompt/PowerShell trên Windows hoặc terminal trên Linux:

```
# Kiểm tra driver NVIDIA và phiên bản CUDA driver đang hỗ trợ
nvidia-smi

# Kiểm tra phiên bản CUDA Toolkit đã cài
nvcc --version
```

Lệnh `nvidia-smi` hiển thị tên GPU cùng phiên bản CUDA cao nhất mà driver hỗ trợ, còn `nvcc --version` hiển thị đúng phiên bản CUDA Toolkit đang cài trên máy. Hai con số này không nhất thiết trùng nhau, chỉ cần bản CUDA Toolkit không vượt quá bản driver hỗ trợ là chạy được.

2.3 Cài đặt thư viện Python

Tôi tạo một virtual environment riêng bằng `uv` để không ảnh hưởng tới Python hệ thống, rồi cài hai thư viện chính là `transformers` và `sentence-transformers`.

```
# Tạo virtual environment với Python 3.12
uv venv --python 3.12 .venv
source .venv/bin/activate

# Cài transformers và sentence-transformers
uv pip install transformers sentence-transformers
```

Kết quả rút gọn, chỉ giữ lại các gói chính:

```
+ huggingface-hub==1.26.1
+ numpy==2.5.1
+ safetensors==0.8.0
+ sentence-transformers==5.7.0
+ tokenizers==0.22.2
+ torch==2.13.0
+ transformers==5.14.1
```

Bạn để ý là `torch` đã tự động được cài kèm dù tôi không khai báo trực tiếp, vì `sentence-transformers` khai báo `torch` là dependency bắt buộc. Tôi kiểm tra lại

xem bản `torch` này đã nhận đúng backend tăng tốc của máy chưa:

check_env.py

```
import torch, transformers, sentence_transformers

print("torch", torch.__version__)
print("transformers", transformers.__version__)
print("sentence-transformers", sentence_transformers.__version__)
print("mps available:", torch.backends.mps.is_available())
print("cuda available:", torch.cuda.is_available())
```

```
torch 2.13.0
transformers 5.14.1
sentence-transformers 5.7.0
mps available: True
cuda available: False
```

Máy tôi chạy macOS Apple Silicon nên không cài CUDA, và `torch` tự nhận đúng backend MPS, tức PyTorch có thể tận dụng GPU tích hợp của chip Apple. Với macOS hoặc chỉ chạy CPU, bạn dừng ở đây, không cần cài thêm gì nữa.

Nếu bạn chạy Windows hoặc Linux và đã cài driver cùng CUDA Toolkit ở bước trước, bản `torch` vừa cài theo mặc định ở trên nhiều khả năng chỉ là bản CPU. Bạn cần cài đè lại đúng bản `torch`, `torchvision`, `torchaudio` khớp với bản CUDA Toolkit đã cài, lấy đúng lệnh tại trang chủ PyTorch:

```
# Ví dụ cài torch cho CUDA 12.4, thay đúng mã CUDA của máy
uv pip install torch torchvision torchaudio \
  --index-url https://download.pytorch.org/whl/cu124
```

> **Lưu ý** Mã CUDA trong đường dẫn trên, ví dụ `cu124`, phải khớp với bản CUDA Toolkit bạn đã cài ở bước trước. Trang chủ PyTorch tại `https://pytorch.org/get-started/locally` luôn cập nhật đúng lệnh cài theo từng bản CUDA, bạn nên đối chiếu lại trước khi chạy thay vì chép nguyên lệnh mẫu ở trên.

3. Xây dựng mã nguồn pipeline

Trước khi viết mã nguồn, tôi cài thêm hai thư viện dùng để kết nối Qdrant và đọc biến môi trường từ file `.env`.

```
uv pip install qdrant-client python-dotenv
```

3.1 Data Class

Tôi định nghĩa hai `dataclass` làm kiểu dữ liệu trả về cho từng giai đoạn của pipeline, thay vì trả về tuple hay dict rời rạc. `RetrievalResult` lưu kết quả thô từ Qdrant, còn `RerankResult` lưu thêm điểm số sau khi Cross-Encoder chấm lại.

retrieval_pipeline.py

```
import argparse
import json
import os
import re
import time
from dataclasses import asdict, dataclass
from datetime import datetime, timezone

from dotenv import load_dotenv
from qdrant_client import QdrantClient
from sentence_transformers import CrossEncoder, SentenceTransformer

load_dotenv()

BI_ENCODER_MODEL = "BAAI/bge-m3"
CROSS_ENCODER_MODEL = "BAAI/bge-reranker-v2-m3"
OUTPUT_DIR = "outputs"
COLLECTION_NAME = os.environ.get("QDRANT_COLLECTION", "documents")

bi_encoder = SentenceTransformer(BI_ENCODER_MODEL)
cross_encoder = CrossEncoder(CROSS_ENCODER_MODEL)


@dataclass
class RetrievalResult:
    doc_id: str
    text: str
    score: float


@dataclass
class RerankResult:
    doc_id: str
    text: str
    retrieval_score: float
    rerank_score: float
```

3.2 Retriever và Reranker

`Retriever` đóng gói kết nối tới Qdrant và bước encode query bằng bi-encoder. Credential đọc từ hai biến môi trường `QDRANT_URL` và `QDRANT_API_KEY`, đã được `load_dotenv()` nạp sẵn vào `os.environ` ở bước trước.

retrieval_pipeline.py

```
class Retriever:
    def __init__(self, collection_name: str, top_k: int = 20):
        self.collection_name = collection_name
        self.top_k = top_k
        self.client = QdrantClient(
            url=os.environ["QDRANT_URL"],
            api_key=os.environ["QDRANT_API_KEY"],
        )

    def search(self, query: str) -> list[RetrievalResult]:
        query_vector = bi_encoder.encode(query).tolist()
        response = self.client.query_points(
            collection_name=self.collection_name,
            query=query_vector,
            limit=self.top_k,
        )
        return [
            RetrievalResult(doc_id=str(point.id), text=point.payload["text"], score=point.
    score)
            for point in response.points
        ]
```

Phương thức `search` chỉ encode query một lần, còn vector của document đã được tính sẵn và lưu trong Qdrant từ trước, đúng với cơ chế Bi-Encoder đã nói ở phần cơ sở lý thuyết. `query_points` trả về tối đa `top_k` điểm có điểm tương đồng cao nhất, tôi đọc lại `payload["text"]` để lấy nội dung document gốc đã lưu kèm vector.

`Reranker` nhận danh sách ứng viên từ `Retriever`, ghép từng document với query thành cặp, rồi đưa nguyên danh sách cặp vào `cross_encoder.predict` một lần thay vì gọi lặp lại cho từng cặp. Đây chính là batch inference, kỹ thuật gom nhiều input thành một lô để mô hình xử lý song song thay vì xử lý tuần tự từng phần tử.

retrieval_pipeline.py

```
class Reranker:
    def __init__(self, top_n: int = 5):
        self.top_n = top_n

    def rerank(self, query: str, candidates: list[RetrievalResult]) -> list[RerankResult]:
        pairs = [(query, candidate.text) for candidate in candidates]
        scores = cross_encoder.predict(pairs)
        ranked = sorted(zip(candidates, scores), key=lambda pair: pair[1], reverse=True)
        return [
            RerankResult(
                doc_id=candidate.doc_id,
                text=candidate.text,
                retrieval_score=candidate.score,
                rerank_score=float(score),
            )
            for candidate, score in ranked[: self.top_n]
        ]
```

`cross_encoder.predict(pairs)` tự chia `pairs` thành các batch nhỏ hơn theo tham số `batch_size` mặc định của `CrossEncoder`, chạy song song trong từng batch thay vì từng cặp riêng lẻ, nên nhanh hơn đáng kể so với việc gọi `predict` nhiều lần trong vòng lặp. Kết quả trả về được sắp xếp giảm dần theo `rerank_score` và chỉ giữ lại `top_n` phần tử đầu.

3.3 Main function

Các class ở trên chỉ định nghĩa hành vi, tôi cần thêm một hàm `main` để biến toàn bộ thành một script chạy thẳng từ dòng lệnh, nhận vào query cùng hai tham số `top_k` và `top_n` từ dòng lệnh.

Trước tiên, tôi viết thêm ba hàm phụ trợ: một hàm rút gọn text khi in ra màn hình, một hàm in kết quả đúng định dạng số thứ tự - text rút gọn - score, và một hàm lưu toàn bộ kết quả ra file JSON đặt tên theo timestamp và theo chính query đó.

retrieval_pipeline.py

```
def truncate(text: str, max_length: int = 80) -> str:
    text = text.replace("\n", " ").strip()
    if len(text) <= max_length:
        return text
    return text[: max_length - 3] + "..."


def print_results(title: str, results: list, score_field: str) -> None:
    print(f"\n{title}")
    for rank, result in enumerate(results, start=1):
        score = getattr(result, score_field)
        print(f"{rank} - {truncate(result.text)} - {score:.4f}")


def slugify(text: str) -> str:
    slug = re.sub(r"[^\w]+", "-", text.lower()).strip("-")
    return slug[:50] or "query"


def save_json(
    query: str,
    retrieval_results: list[RetrievalResult],
    rerank_results: list[RerankResult],
    retrieve_seconds: float,
    rerank_seconds: float,
) -> str:
    os.makedirs(OUTPUT_DIR, exist_ok=True)
    timestamp = datetime.now(timezone.utc).strftime("%Y%m%d-%H%M%S")
    output_path = os.path.join(OUTPUT_DIR, f"{timestamp}_{slugify(query)}.json")
    top1_changed = (
        bool(retrieval_results)
        and bool(rerank_results)
        and retrieval_results[0].doc_id != rerank_results[0].doc_id
    )
```

```
    payload = {
        "query": query,
        "timing": {
            "retrieve_seconds": retrieve_seconds,
            "rerank_seconds": rerank_seconds,
            "total_seconds": retrieve_seconds + rerank_seconds,
        },
        "quality": {
            "top1_changed": top1_changed,
        },
        "retrieval": [asdict(result) for result in retrieval_results],
        "rerank": [asdict(result) for result in rerank_results],
    }
    with open(output_path, "w", encoding="utf-8") as f:
        json.dump(payload, f, ensure_ascii=False, indent=2)
    return output_path
```

Cuối cùng là hàm `main`, nơi tôi khai báo tham số dòng lệnh, khởi tạo `Retriever` và `Reranker`, rồi chạy nối tiếp hai bước retrieve và rerank. Tôi bọc riêng từng bước bằng `time.perf_counter()` để đo đúng phần thời gian Cross-Encoder cộng thêm so với việc chỉ dùng Bi-Encoder, và so `doc_id` đứng đầu trước/sau rerank để xem chất lượng có thay đổi hay không:

retrieval_pipeline.py

```
def main() -> None:
    parser = argparse.ArgumentParser(
        description="Run the retrieve-then-rerank pipeline for a single query."
    )
    parser.add_argument("query", type=str, help="Query to search for")
    parser.add_argument(
        "--top-k",
        type=int,
        default=20,
        help="Number of candidates to fetch in the retrieve step (Retriever)",
    )
    parser.add_argument(
        "--top-n",
        type=int,
        default=5,
        help="Number of results to keep after the rerank step (Reranker)",
    )
    args = parser.parse_args()

    retriever = Retriever(collection_name=COLLECTION_NAME, top_k=args.top_k)
    reranker = Reranker(top_n=args.top_n)

    print(f"Query: {args.query}")

    retrieve_start = time.perf_counter()
    retrieval_results = retriever.search(args.query)
    retrieve_seconds = time.perf_counter() - retrieve_start
    print_results("Results after retrieve:", retrieval_results, "score")
```

```
    rerank_start = time.perf_counter()
    rerank_results = reranker.rerank(args.query, retrieval_results)
    rerank_seconds = time.perf_counter() - rerank_start
    print_results("Results after rerank:", rerank_results, "rerank_score")

    total_seconds = retrieve_seconds + rerank_seconds
    top1_changed = (
        bool(retrieval_results)
        and bool(rerank_results)
        and retrieval_results[0].doc_id != rerank_results[0].doc_id
    )

    print("\nTiming:")
    print(f"  retrieve: {retrieve_seconds:.3f}s")
    print(f"  rerank:   {rerank_seconds:.3f}s ({rerank_seconds / total_seconds:.0%} of
    total)")
    print(f"  total:    {total_seconds:.3f}s")
    print(f"Top-1 changed after rerank: {top1_changed}")

    output_path = save_json(
        args.query, retrieval_results, rerank_results, retrieve_seconds, rerank_seconds
    )
    print(f"\nSaved results to {output_path}")


if __name__ == "__main__":
    main()
```

Bước rerank gần như luôn chiếm phần lớn `total_seconds`, đúng với độ phức tạp $O(N \cdot L^2)$ đã tính ở phần cơ sở lý thuyết, vì Cross-Encoder phải chạy riêng một lượt forward pass cho từng ứng viên thay vì tra cứu chỉ mục có sẵn như Bi-Encoder. `top1_changed` chỉ là một tín hiệu chất lượng nhanh cho từng query đơn lẻ, còn con số Hit@k và MRR tổng hợp trên nhiều query tôi sẽ tính ở phần đánh giá kết quả tiếp theo.

4. Đánh giá kết quả

4.1 Môi trường kiểm thử

Toàn bộ phần đánh giá dưới đây chạy trên cùng máy và cùng môi trường `uv` đã dựng ở phần chuẩn bị môi trường:

```
# Kiểm tra lại phiên bản Python trong virtual environment và phiên bản uv
python3 --version && uv --version
```

```
Python 3.12.9
uv 0.10.6 (Homebrew 2026-02-24)
```

Về dữ liệu, bài trước đó đã hướng dẫn bạn tạo một collection Qdrant tên

`legal_documents` với dữ liệu sample là Luật Đất đai số 31/2024/QH15. Collection đó phù hợp để làm quen với pipeline, nhưng để kiểm chứng đúng chất lượng của Bi-Encoder và Cross-Encoder, tôi cần một bộ dữ liệu đa dạng hơn, phản ánh đúng thực tế nhiều loại văn bản pháp luật cùng tồn tại và chồng lấn nội dung lên nhau. Vì vậy tôi đã chuẩn bị sẵn một collection khác tên `VBPL` trên một Qdrant chạy local bằng Docker, gồm nhiều luật, nghị định và thông tư khác nhau đã được chunk và encode sẵn.

Tôi thêm hai dòng sau vào file `.env` để chương trình đọc đúng địa chỉ và tên collection cần dùng:

```
QDRANT_URL=http://localhost:6333
QDRANT_COLLECTION=VBPL
```

Trước khi chạy đánh giá, tôi kiểm tra lại số lượng point thật sự có trong collection để chắc chắn dữ liệu đã nạp đầy đủ:

```
# Kiểm tra số lượng point trong collection VBPL
curl -s http://localhost:6333/collections/VBPL | python3 -m json.tool | grep points_count
```

```
    "points_count": 14326,
```

4.2 Kết quả kiểm thử

Để kiểm tra chất lượng của retrieval khi có thêm bước rerank, với hai metric Hit@k và MRR, tôi đã chuẩn bị sẵn bộ 10 câu hỏi sau cùng điều luật ground-truth của chúng.

1. Con tôi năm nay 16 tuổi, hè này cháu xin vào phụ quán cà phê gần nhà làm buổi tối, tôi hơi lo không biết công việc kiểu gì thì phù hợp cho tuổi cháu, có quy định nào giới hạn việc này không? *(Đáp án đúng: Bộ luật Lao động 2019, Điều 144 khoản 1)*
2. Năm ngoái một đơn vị nước ngoài đặt tại Việt Nam phụ trách sổ sách và báo cáo cuối năm cho công ty tôi, giờ tôi định nhờ luôn bên đó xác nhận lại toàn bộ báo cáo đó có đúng chuẩn không, liệu có vướng mắc gì về mặt pháp lý không? *(Đáp án đúng: Luật Kiểm toán độc lập 2011, Điều 30 khoản 2 điểm a)*
3. Bên tôi định góp một mảnh đất để làm sân bãi phục vụ cho việc chở hàng bằng máy bay, nhưng chưa biết phải chứng minh giá trị mảnh đất đó bằng giấy tờ gì cho hợp lệ? *(Đáp án đúng: Nghị định 30/2013/NĐ-CP, Điều 9 khoản 2)*
4. Xưởng sản xuất của tôi để hoá chất rò rỉ ra con suối cạnh đó, mà bên tôi chưa từng chuẩn bị kế hoạch xử lý cho tình huống này, nếu bị kiểm tra thì có bị xử lý gì không? *(Đáp án đúng: Nghị định 36/2020/NĐ-CP, Điều 24 khoản 1 điểm a)*
5. Nhà xưởng của công ty tôi đứng tên chung với một đối tác khác, giờ bị cơ quan thi

hành án niêm phong để bán, đối tác kia có được quyền gì đặc biệt khi tài sản đem ra bán không? *(Đáp án đúng: Nghị định 44/2020/NĐ-CP, Điều 33 khoản 4)*

6. Tôi đánh bài ở một sòng bài và thắng lớn nhưng đến giờ vẫn chưa nhận được tiền, bên đó cứ hẹn tới hẹn lui, phía kinh doanh có trách nhiệm gì trong việc trả tiền cho khách không? *(Đáp án đúng: Nghị định 03/2017/NĐ-CP, Điều 21 khoản 2 điểm d)*
7. Tôi đang cân nhắc tham gia bảo hiểm theo diện không bắt buộc, nghe nói cách tính tiền đóng hàng tháng khác với người đi làm công ty, không biết người ta lấy con số nào làm cơ sở để tính? *(Đáp án đúng: Luật Bảo hiểm xã hội 2024, Điều 5 khoản 2)*
8. Cơ quan tôi sắp vay tiền nước ngoài để làm dự án nhưng hiệp định vay chưa ký xong, bên tư vấn bảo có thể chuẩn bị hồ sơ chọn nhà thầu từ bây giờ, việc này áp dụng cho loại dự án nào? *(Đáp án đúng: Luật Đấu thầu 2023, Điều 42 khoản 1)*
9. Tôi mới xịt thuốc trừ sâu cho vườn rau nhà mình xong, không biết phải chờ bao lâu mới được hái rau ăn cho an toàn, khái niệm đó luật gọi là gì? *(Đáp án đúng: Luật Bảo vệ và kiểm dịch thực vật 2013, Điều 3 khoản 22)*
10. Tôi có mua một ít sách tiếng nước ngoài gửi về nhà để đọc, không kinh doanh gì cả, nộp hồ sơ xin phép rồi thì cơ quan nhà nước mất bao lâu để trả lời? *(Đáp án đúng: Luật Xuất bản 2012, Điều 41 khoản 4)*

> **Lưu ý** `Reranker.rerank()` luôn chấm điểm lại toàn bộ ứng viên nhận vào rồi mới cắt theo `top_n`, nên nếu chạy với `top_n=5` mặc định, tôi sẽ không thấy được đáp án đúng rơi xuống vị trí nào ngoài top-5. Vì vậy khi đánh giá, tôi gọi `retrieval_pipeline.py` với cả `top_k` lẫn `top_n` cùng bằng 20 để xem được toàn bộ danh sách sau rerank, còn Hit@k và MRR báo cáo bên dưới vẫn tính theo top-5.

Lệnh tôi đã dùng để chạy pipeline:

```
python retrieval_pipeline.py \
  "Con tôi năm nay 16 tuổi, hè này cháu xin vào phụ quán cà phê gần nhà làm
  buổi tối, tôi hơi lo không biết công việc kiểu gì thì phù hợp cho tuổi
  cháu, có quy định nào giới hạn việc này không?" \
  --top-k 20 \
  --top-n 20
```

Output rút gọn của câu 1, chỉ giữ lại dòng chứa đáp án đúng trong hai danh sách và phần tổng kết:

```
Query: Con tôi năm nay 16 tuổi, hè này cháu xin vào phụ quán cà phê gần nhà
làm buổi tối, tôi hơi lo không biết công việc kiểu gì thì phù hợp cho tuổi
cháu, có quy định nào giới hạn việc này không?

Results after retrieve:
```

```
...
6 - Điều 144. Nguyên tắc sử dụng lao động chưa thành niên... - 0.6841
...

Results after rerank:
...
8 - Điều 144. Nguyên tắc sử dụng lao động chưa thành niên... - 0.7234
...

Timing:
  retrieve: 0.996s
  rerank:   1.297s (57% of total)
  total:    2.293s
Top-1 changed after rerank: True

Saved results to outputs/20260406-142107_con-toi-nam-nay-16-tuoi.json
```

Tôi lặp lại đúng lệnh trên, luôn với `top_k=20` và `top_n=20`, cho cả 10 câu, mỗi câu ghi lại vị trí của đáp án đúng trong danh sách sau bước retrieve và trong danh sách sau bước rerank, tính trên toàn bộ 20 ứng viên chứ không chỉ 5 vị trí đầu. Dấu "–" nghĩa là đáp án đúng không nằm trong 20 ứng viên đó, dù ở bước nào:

Câu	Đáp án đúng	Rank trước	Rank sau	Retrieve (s)	Rerank (s)
1	Bộ luật Lao động 2019, Đ.144	6	8	0.996	1.297
2	Luật Kiểm toán độc lập 2011, Đ.30	-	-	0.284	2.273
3	Nghị định 30/2013/NĐ-CP, Đ.9	-	-	0.269	1.968
4	Nghị định 36/2020/NĐ-CP, Đ.24	-	-	0.291	1.450
5	Nghị định 44/2020/NĐ-CP, Đ.33	3	2	0.440	1.923
6	Nghị định 03/2017/NĐ-CP, Đ.21	5	5	0.154	1.554
7	Luật Bảo hiểm xã hội 2024, Đ.5	14	4	0.121	2.214
8	Luật Đấu thầu 2023, Đ.42	-	-	0.164	4.388
9	Luật Bảo vệ và kiểm dịch thực vật 2013, Đ.3	1	1	0.129	5.218
10	Luật Xuất bản 2012, Đ.41	5	1	0.181	1.486
	Trung bình			*0.303*	*2.377*

Hit@5 và MRR bên dưới đều tính theo top-5, tức chỉ những đáp án có rank từ 1 đến 5 trong bảng trên mới được coi là "trúng"; câu 1 dù thấy rõ đáp án đúng vẫn còn trong 20 ứng viên ở hạng 8 sau rerank, khi tính Hit@5/MRR vẫn bị coi là trượt như những câu không tìm thấy đáp án trong cả 20 ứng viên:

Chỉ số	Trước rerank	Sau rerank
Hit@5	4/10 (40%)	5/10 (50%)
MRR	0.197	0.308

Bốn trên mười câu (câu 2, 3, 4, 8) không có đáp án đúng trong cả 20 ứng viên ngay từ bước retrieve, nên dù rerank có tốt đến đâu cũng không cứu được, vì Cross-Encoder chỉ sắp xếp lại đúng những gì Bi-Encoder đã đưa lên chứ không tự tìm thêm ứng viên mới. Đây chính là giới hạn về recall của Bi-Encoder khi câu hỏi diễn đạt theo tình huống đời thường thay vì lặp lại từ khoá pháp lý.

Một khả năng khác cũng đáng nghi cho cả nhóm "không lọt top-20" lẫn 6 câu còn lại: corpus `VBPL` lưu cùng lúc văn bản luật gốc, văn bản hợp nhất và các nghị định, thông tư hướng dẫn thi hành, nên nhiều chunk có nội dung na ná nhau vẫn có thể cạnh tranh vị trí với chunk ground-truth. Tôi lấy nguyên văn top ứng viên của cả 6 câu tìm thấy trong top-20 để kiểm tra khả năng này thay vì đoán.

Với câu 2 (đã xét ở câu "ngoài top-20" phía trên), phần lớn vị trí đầu bị chiếm bởi các chunk thuộc Văn bản hợp nhất 11/VBHN-VPQH năm 2015 và 17/VBHN-VPQH năm 2025, đều là bản hợp nhất của chính Luật Kiểm toán độc lập 2011 — khớp đúng giả thuyết. Câu 7 cũng khớp rõ: trước rerank, ba vị trí đầu là Nghị định 157/2025/NĐ-CP và hai chunk Thông tư 37/2017/TT-BQP, đều là văn bản hướng dẫn cách tính mức đóng bảo hiểm xã hội bắt buộc riêng cho quân nhân, công an — cùng khái niệm "mức đóng tính trên cơ sở nào" như câu hỏi nhưng áp dụng cho một nhóm đối tượng hẹp hơn, khiến đáp án đúng (nguyên tắc chung ở Luật Bảo hiểm xã hội 2024) bị đẩy xuống tận rank 14; rerank nhận ra câu hỏi không nhắc gì đến quân nhân hay công an nên kéo đúng chunk lên rank 4. Câu 1 cũng có một văn bản hướng dẫn đứng đầu bảng, cụ thể Thông tư 09/2020/TT-BLĐTBXH hướng dẫn thi hành các quy định về lao động chưa thành niên của Bộ luật Lao động, nhưng ở đây hiện tượng lại phản tác dụng: rerank đánh giá thông tư hướng dẫn cùng vài điều lân cận (Điều 143, 146, 147) sát nghĩa với câu hỏi hơn cả đáp án đúng, nên đáp án đúng còn tụt từ rank 6 xuống rank 8.

Ngược lại, ba câu 5, 6, 10 không cho thấy dấu hiệu của hiện tượng này. Các chunk đứng trước đáp án đúng ở những câu này thuộc điều luật khác, nghị định khác, hoặc lĩnh vực khác hẳn (ví dụ câu 6 có chunk về ngoại hối trong casino và về đặt cược đua ngựa, câu 10 có chunk về nhập khẩu hàng hoá theo Luật Quản lý ngoại thương), chỉ đơn thuần trùng một vài từ khoá chứ không phải nội dung gần giống ground-truth. Câu 9 thì đáp án đúng đã đứng rank 1 ngay từ đầu, không có gì để phân tích thêm.

Giới hạn của phương pháp đánh giá Vì Hit@k và MRR ở đây tính theo đúng một `doc_id` duy nhất, corpus `VBPL` dư thừa nhiều bản sao gần giống nhau của cùng một quy định (văn bản luật gốc, văn bản hợp nhất, nghị định và thông tư hướng dẫn thi hành) khiến số liệu không phản ánh đúng khả năng thật của hệ thống, theo hai chiều trái ngược. Ở câu 2, các bản sao này lấn hết chỗ trong top-20 khiến chunk ground-truth bị tính là trượt hoàn toàn dù hệ thống vẫn trả về đúng nội dung pháp lý ở một văn bản khác. Ở câu 1, một văn bản hướng dẫn lại được Cross-Encoder chấm sát nghĩa hơn cả ground-truth nên đáp án đúng còn bị đẩy xuống thấp hơn sau rerank, dù nội dung đôi bên gần như tương đương. Vì cả hai chiều lệch đều xuất phát từ cùng một nguyên nhân, Hit@k và MRR trong phần này nên được đọc như thước đo có nhiễu do cấu trúc corpus, không phải con số tuyệt đối phản ánh chất lượng mô hình.

5. Vấn đề bộ nhớ và độ trễ của pipeline

5.1 Vì sao rerank dễ gặp OOM

OOM (Out Of Memory) là lỗi xảy ra khi chương trình cố cấp phát nhiều bộ nhớ hơn phần cứng còn trống, có thể là RAM hệ thống hoặc VRAM của GPU. Với pipeline retrieve rồi rerank, OOM gần như luôn xuất hiện ở bước rerank chứ không phải retrieve, vì hai bước này tiêu tốn bộ nhớ theo cách rất khác nhau.

Ở bước retrieve, Bi-Encoder chỉ encode một query duy nhất mỗi lần gọi, còn vector của toàn bộ document đã được tính sẵn và lưu trong Qdrant từ trước. Bộ nhớ cần dùng gần như cố định, không phụ thuộc vào số lượng document trong collection.

Ở bước rerank, Cross-Encoder phải ghép query với từng document ứng viên thành một cặp, đưa cả cặp qua toàn bộ mô hình. Khi gọi `cross_encoder.predict(pairs)`, thư viện tự cắt `pairs` thành nhiều batch nhỏ hơn theo tham số `batch_size`, mặc định là 32 theo tài liệu chính thức của Sentence Transformers cho `CrossEncoder.predict`[1], và xử lý tuần tự từng batch chứ không gộp hết `pairs` vào một lần forward pass duy nhất; `bi_encoder.encode()` ở bước retrieve cũng dùng chung mặc định `batch_size=32` này[2]. Bộ nhớ cần dùng cho một batch tăng xấp xỉ theo tích `batch_size` × `sequence_length`^{2}, vì cơ chế attention bên trong Transformer so sánh từng token với mọi token khác trong cùng chuỗi, còn `sequence_length` là tổng độ dài token của cặp (query cộng document) sau khi đã ghép và tokenize. Vì `predict()` luôn cắt theo `batch_size` cố định, tăng tổng số ứng viên đưa vào rerank (`top_k`) không tự làm một batch phình to hơn, mà chỉ làm tăng số batch phải chạy nối tiếp, tức tăng độ trễ và tổng lượng tính toán chứ không phải bộ nhớ đỉnh tại một thời điểm; ngược lại, tăng

[1] https://sbert.net/docs/package_reference/cross_encoder/model.html
[2] https://sbert.net/docs/package_reference/sentence_transformer/model.html

batch_size hoặc sequence_length mới trực tiếp làm bộ nhớ đỉnh tăng lên.

5.2 Ước lượng VRAM và chọn batch_size phù hợp

Vì bộ nhớ của rerank phụ thuộc trực tiếp vào batch_size và sequence_length chứ không phải top_k, tôi ước lượng trước mức VRAM cần dùng theo batch_size bằng một công thức xấp xỉ cho phần tốn bộ nhớ nhất trong Cross-Encoder: ma trận điểm attention. Chuẩn attention thông thường (chưa tối ưu như FlashAttention) cần lưu nguyên ma trận điểm attention kích thước sequence_length × sequence_length cho mỗi attention head, tức độ phức tạp bộ nhớ $O(N^2)$ theo sequence_length, như bài FlashAttention của Dao và cộng sự đã chỉ ra[3]. Từ đó suy ra công thức xấp xỉ cho riêng phần này:

$$\text{VRAM (byte)} \approx \text{batch_size} \times \text{num_heads} \times \text{sequence_length}^2 \times \text{bytes_per_phần_tử}$$

bge-reranker-v2-m3 dùng chung backbone kiến trúc với bge-m3 (XLM-RoBERTa-large), có 16 attention head, còn sequence_length mặc định của bge-m3 là 2048 token. Đây là công thức xấp xỉ: nó chỉ tính phần lưu ma trận attention, chưa gồm trọng số model, các hidden state trung gian khác và overhead của PyTorch/CUDA allocator, nên số thực tế đo được thường khác con số này, chỉ nên dùng để ước lượng bậc độ lớn (order of magnitude) chứ không phải giá trị chính xác. Với sequence_length=2048 cố định và giả sử chạy ở độ chính xác fp16 (2 byte/phần tử), VRAM ước lượng theo từng mức batch_size:

batch_size	VRAM ước lượng (attention, fp16)
8	≈ 1 GB
16	≈ 2 GB
32	≈ 4 GB
64	≈ 8 GB
128	≈ 16 GB
256	≈ 32 GB

Đảo ngược công thức, trừ hao khoảng 1.5GB VRAM cho trọng số model và overhead của framework, cho ra mức batch_size nên chọn theo từng loại GPU phổ biến, vẫn với sequence_length=2048:

VRAM GPU	batch_size gợi ý
6 GB	≈ 32
8 GB	≈ 48

[3] https://arxiv.org/abs/2205.14135

VRAM GPU	batch_size gợi ý
12 GB	≈ 80
16 GB	≈ 112
24 GB	≈ 176

`batch_size` mặc định 32 của thư viện vừa khớp với ngưỡng an toàn cho một GPU 6GB, còn muốn tận dụng GPU lớn hơn thì tăng `batch_size` theo bảng trên thay vì giữ nguyên mặc định. Bảng này cũng cho biết ngược lại: nếu cố tình đặt `batch_size` vượt xa mức gợi ý cho GPU đang dùng, ví dụ vài trăm hoặc vài nghìn, khả năng gặp lỗi `CUDA out of memory` là rất cao.

Ba hướng khắc phục khi gặp OOM ở bước rerank:

Giảm batch_size khi gọi predict Đây là biện pháp trực tiếp nhất để giảm bộ nhớ cần dùng tại mỗi lượt forward pass, chọn theo bảng gợi ý ở trên cho đúng loại GPU đang dùng.

Giới hạn top_k trước khi rerank Không trực tiếp chống OOM vì `predict()` đã tự cắt batch theo `batch_size`, nhưng `top_k` càng lớn thì càng nhiều batch phải chạy nối tiếp, tốn thời gian và tổng lượng tính toán không cần thiết. Chỉ đưa vào rerank đúng số ứng viên cần thiết, thường vài chục là đủ.

Dùng model nhẹ hơn hoặc chạy trên CPU Nếu không có GPU đủ VRAM ngay cả với `batch_size` nhỏ, đổi sang một Cross-Encoder nhỏ hơn hoặc chấp nhận chạy CPU chậm hơn nhưng không giới hạn bởi VRAM.

Trường hợp OOM thứ hai hay gặp là khi encode toàn bộ corpus lớn để đưa vào Qdrant lúc đầu, ví dụ load hết vài triệu document vào một list rồi gọi `bi_encoder.encode()` một lần duy nhất. `encode()` cũng tự cắt batch theo `batch_size` mặc định 32 như `predict()` nên bản thân bước forward pass không phải vấn đề; nguyên nhân OOM ở trường hợp này thường nằm ở việc giữ nguyên toàn bộ text gốc và vector kết quả của vài triệu document trong một list Python cùng lúc, tốn RAM hệ thống chứ không phải VRAM. Cách xử lý: chia corpus thành từng lô nhỏ, encode và upload lên Qdrant theo từng lô bằng một generator thay vì giữ toàn bộ dữ liệu và vector trong RAM cùng lúc.

5.3 Tối ưu latency

Nhìn lại bảng thời gian đã đo ở phần đánh giá kết quả, retrieve trung bình mất 0.303 giây còn rerank trung bình mất 2.377 giây trên 10 câu hỏi, tức rerank chiếm phần lớn tổng thời gian phản hồi. Con số này khớp với độ phức tạp tính toán đã nói ở phần cơ sở lý thuyết: retrieve chỉ tra cứu chỉ mục có sẵn trong Qdrant, còn rerank phải chạy một lượt forward pass Transformer đầy đủ cho từng ứng viên, nên chậm hơn hẳn dù chỉ xử lý top-20 ứng viên.

Vài hướng tối ưu latency cho bước rerank, đánh đổi lấy độ chính xác ở các mức khác nhau:

Giảm top_k đưa vào rerank Rerank 10 ứng viên thay vì 20 giảm gần một nửa thời gian rerank, đổi lại rủi ro bỏ sót ứng viên đúng nếu nó đứng ngoài top-10 sau retrieve.

Dùng Cross-Encoder nhỏ hơn Model càng ít tham số và ít layer thì mỗi forward pass càng nhanh, đổi lại độ chính xác thường thấp hơn model lớn.

Cache kết quả cho query lặp lại Nếu hệ thống có nhiều người dùng hỏi cùng một câu hoặc câu gần giống nhau, lưu lại kết quả rerank theo query để tránh chạy lại từ đầu.

Luôn gọi predict theo batch Như đã nói ở phần pipeline, gọi `cross_encoder.predict(pairs)` một lần cho cả danh sách luôn nhanh hơn gọi lặp lại `predict` cho từng cặp một, vì tận dụng được xử lý song song của phần cứng.

6. Kết luận

Trong bài này, tôi bắt đầu từ hai kiến trúc nền cho bài toán tìm kiếm ngữ nghĩa:

Bi-Encoder mã hoá query và document độc lập nên tra cứu nhanh bằng vector có sẵn, nhưng vì không thấy được tương tác trực tiếp giữa hai bên nên độ chính xác có giới hạn.

Cross-Encoder mã hoá đồng thời cả cặp nên nắm được tương tác đó, chính xác hơn hẳn nhưng phải chạy riêng một forward pass cho từng ứng viên nên chậm hơn nhiều bậc.

Một quy trình phổ biến là retrieve bằng Bi-Encoder để thu hẹp nhanh xuống một tập nhỏ ứng viên, rồi rerank bằng Cross-Encoder để chấm lại kỹ trong phạm vi đó, thay vì bắt Cross-Encoder xử lý toàn bộ collection.

Để kiểm chứng, tôi chạy pipeline này trên 10 câu hỏi tình huống thật, đối chiếu với điều luật ground-truth đã biết trước trong collection `VBPL`. Kết quả đo được: Hit@5 và MRR đều tăng khi thêm rerank. Rerank giúp ích rõ với những câu mà Bi-Encoder tìm đúng tài liệu nhưng xếp chưa cao, còn với những câu Bi-Encoder không tìm ra tài liệu đúng trong top-20 ngay từ đầu thì rerank cũng chịu, vì nó chỉ sắp lại chứ không tự tìm thêm ứng viên mới.

Với quy mô thực nghiệm chỉ 10 câu hỏi trên một corpus cụ thể, tôi không dám khẳng định rerank luôn đáng giá trong mọi trường hợp. Nhưng qua những gì quan sát được, rerank có vẻ đáng thêm vào khi corpus chứa nhiều document nội dung gần giống nhau (như văn bản luật gốc, bản hợp nhất, nghị định hướng dẫn cùng nói một quy định) khiến một mình Bi-Encoder khó phân biệt, hoặc khi hệ thống chấp nhận đánh đổi thêm một, hai giây độ trễ để lấy độ chính xác cao hơn. Ngược lại, nếu corpus nhỏ và câu hỏi đơn giản để Bi-Encoder một mình đã đủ tốt, hoặc ứng dụng cần phản hồi gần như tức thì, hoặc hạ

tầng không đủ RAM/GPU để chạy thêm một mô hình Transformer nữa, thì rerank chưa hẳn là bước cần thêm ngay.

AI Guru

AI Guru

AI Guru

Nguồn tham khảo

1. Sentence Transformers – Retrieve & Re-Rank: https://www.sbert.net/examples/applications/retrieve_rerank/README.html
2. Sentence Transformers – Cross-Encoder: https://www.sbert.net/examples/applications/cross-encoder/README.html
3. Model card BAAI/bge-m3 (bi-encoder dùng trong bài): https://huggingface.co/BAAI/bge-m3
4. Model card BAAI/bge-reranker-v2-m3 (cross-encoder dùng trong bài): https://huggingface.co/BAAI/bge-reranker-v2-m3
5. Qdrant – Search API và query_points: https://qdrant.tech/documentation/concepts/search/
6. Qdrant – Snapshots (backup và restore collection): https://qdrant.tech/documentation/concepts/snapshots/
7. PyTorch – CUDA memory management: https://pytorch.org/docs/stable/notes/cuda.html#memory-management
8. Sentence Transformers – CrossEncoder.predict (batch_size mặc định): https://sbert.net/docs/package_reference/cross_encoder/model.html#sentence_transformers.cross_encoder.model.CrossEncoder.predict
9. Sentence Transformers – SentenceTransformer.encode (batch_size mặc định): https://sbert.net/docs/package_reference/sentence_transformer/model.html#sentence_transformers.sentence_transformer.model.SentenceTransformer.encode
10. Dao et al. – FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness (nguồn cho độ phức tạp bộ nhớ $O(N^2)$ của attention chuẩn): https://arxiv.org/abs/2205.14135